El-Gamal Digital Signature Scheme

Course Title: Information Security Laboratory
Course No.: 0714 02 CSE 4108

Submitted to,

Dr. Anupam Kumar Bairagi
Professor,
Computer Science and Engineering Discipline,
Khulna University, Khulna-9208

Submitted by,

Md. Siam Ahmed
Student ID: 230202
Year: 4th, Term: 1st
Computer Science and Engineering Discipline
Khulna University, Khulna-9208

The Implementation of El-Gamal Digital Signature Scheme:

```
#include <iostream>
#include <string>
#include <vector>
#include <windows.h>
#include <bcrypt.h>
#include <cstdint>
#include <random>
#ifndef NT_SUCCESS
#define NT_SUCCESS(Status) (((NTSTATUS)(Status)) >= 0)
#endif
#define int long long
#pragma comment(lib, "Bcrypt.lib")
using namespace std;
uint64_t hash_message_sha512(const std::string &M, uint64_t q)
{
    BCRYPT_ALG_HANDLE hAlg = nullptr;
    BCRYPT_HASH_HANDLE hHash = nullptr;
    DWORD cbObj = 0, dummy = 0;

    std::vector<BYTE> pbHash(64);
    std::vector<BYTE> obj;

    if (!NT_SUCCESS(BCryptOpenAlgorithmProvider(&hAlg, BCRYPT_SHA512_ALGORITHM,
nullptr, 0)) ||
        !NT_SUCCESS(BCryptGetProperty(hAlg, BCRYPT_OBJECT_LENGTH, (PBYTE)&cbObj,
sizeof(cbObj), &dummy, 0)))
    {
        if (hAlg)
            BCryptCloseAlgorithmProvider(hAlg, 0);
        return 0;
    }

    obj.resize(cbObj);

    if (NT_SUCCESS(BCryptCreateHash(hAlg, &hHash, obj.data(), cbObj, nullptr, 0,
0)))
    {
        BCryptHashData(hHash, (PBYTE)M.data(), (ULONG)M.size(), 0);
        BCryptFinishHash(hHash, pbHash.data(), 64, 0);
        BCryptDestroyHash(hHash);
    }
    BCryptCloseAlgorithmProvider(hAlg, 0);
```

```
    uint64_t m = 0;
    for (BYTE b : pbHash)
    {
        m = (m * 256 + b) % q;
    }

    return m;
}

int bin_exp(int a, int b, int mod)
{
    if (b == 0)
        return 1;

    int val = bin_exp(a, b / 2, mod);

    if (b % 2 == 0)
        return (1LL * val * val) % mod;
    else
        return (1LL * val * val % mod * a) % mod;
}
int inv(int a, int b)
{
    int x = 1, y = 0, m = b;
    while (b)
    {
        int q = a / b;
        int64_t temp_x = y;
        y = x - q * y;
        x = temp_x;

        int64_t temp_a = b;
        b = a - q * b;
        a = temp_a;
    }
    return (x + m) % m;
}
uint64_t generate_random_number(uint64_t q)
{
    std::random_device rd;
    std::mt19937_64 gen(rd());
    std::uniform_int_distribution<uint64_t> distrib(1, q - 1);
    return distrib(gen);
}
```

```
int32_t main()
{
    cout << "\n\nEl-Gamal Digital Signature Scheme\n";
    int q, a;
    cout << "Enter a Prime number q (less than 2147483647): ";
    cin >> q;
    cout << "Enter a primitive root a of q: ";
    cin >> a;

    int xa = generate_random_number(q - 1);
    cout << "Private key of A, XA: " << xa << endl;
    int ya = bin_exp(a, xa, q);
    cout << "Public key of A, {q, a, YA}: { " << q << ", " << a << ", " << ya <<
" }.\n";
    string message;
    cout << "Enter the message: ";
    cin >> message;
    int m = hash_message_sha512(message, q);
    cout << "Hash of message: " << m << endl;

    int k;
    while (gcd(q - 1, k = generate_random_number(q)) != 1)
    {
    }
    cout << "K: " << k << endl;
    int s1 = bin_exp(a, k, q);
    cout << "S1: " << s1 << endl;
    int k_inv = inv(k, q - 1);
    cout << "Inverse of K: " << k_inv << endl;
    int s2 = (k_inv * (m - (xa * s1) % (q - 1) + q - 1) % (q - 1)) % (q - 1);
    cout<<"S2: "<<s2<<endl;

    int v1=bin_exp(a,m,q);
    int v2=((bin_exp(ya,s1,q)*bin_exp(s1,s2,q))%q);
    cout<<"V1: "<<v1<<endl;
    cout<<"V2: "<<v2<<endl;
    if(v1==v2)cout<<"The produced signature is valid.\n";
    else cout<<"The produced signature is not valid.\n";

    return 0;
}
```

Input/Output:

